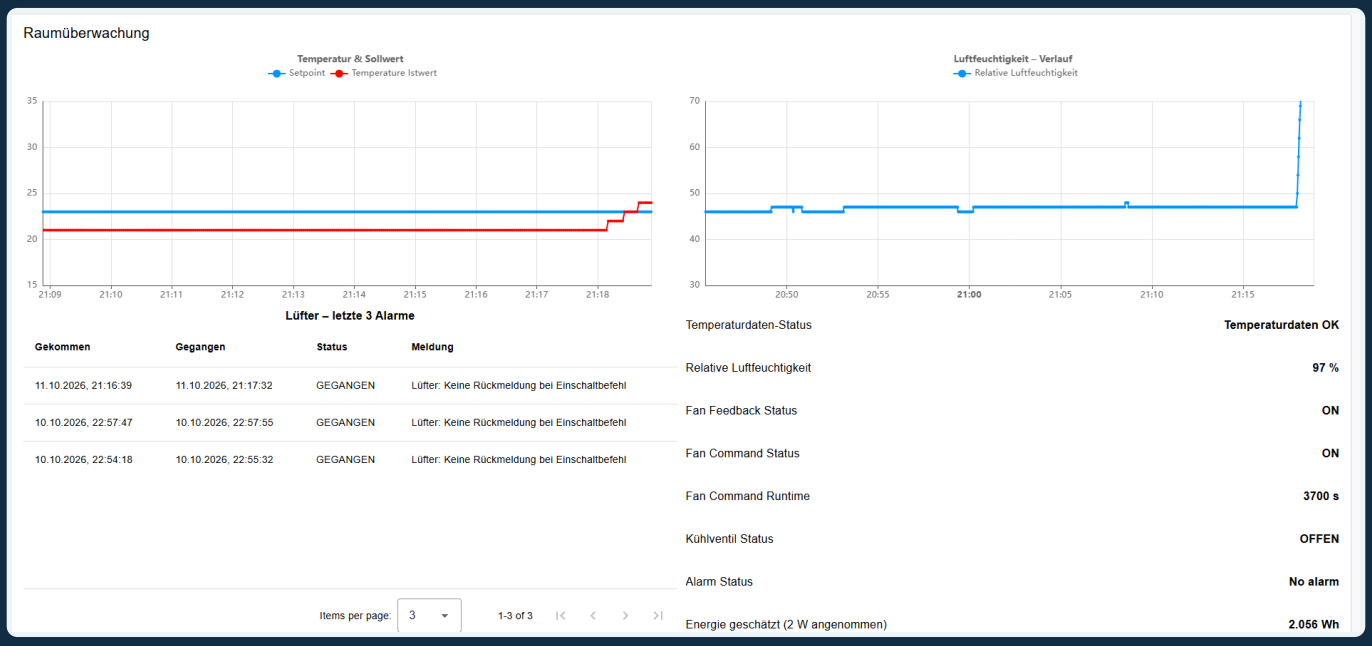
BUILDING AUTOMATION / SYSTEM INTEGRATION

BMS mit ESP32 MQTT & Modbus TCP

Sensorik · Steuerung · Visualisierung · Energie



Ahmad Azroun

Building Automation & Smart Energy Systems

BMS · IoT · System Integration

Physics · Renewable Energy

Technische Dokumentation | Version 1.0 | 11. Oktober 2026

Berlin | github.com/ahmad-energy-iot/BMS-ESP32-MQTT-HVAC

Projekt auf einen Blick

Ein realer ESP32 erfasst Temperatur und relative Luftfeuchtigkeit. Node-RED auf einem Raspberry Pi verarbeitet die Daten, schaltet einen 5-V-Lüfter und steuert eine LED als Kühlventil-Darstellung. Ein Modbus-TCP-Simulator ergänzt Sollwert, Befehl und simulierte Rückmeldung.

Bereich	Ergebnis
Messung	DHT11: Temperatur und Luftfeuchtigkeit, ungefähr alle 2 s
Kommunikation	MQTT-Messwerte und zwei separate Befehlstopics
Regelung	AUTO/MANUAL; Hysterese 24/22 °C bei Sollwert 23 °C
Überwachung	Temperaturdaten-Timeout und verzögerter Rückmeldungsalarm
Dokumentation	Drei Alarmereignisse, Beginn/Ende, Tests und Fehlerdiagnose
Datenhaltung	Persistente ON-Befehlsdauer und Alarmhistorie
Energie	Rechnerische Abschätzung bei angenommenen 2 W

Projektstatus
Am persönlichen Laboraufbau praktisch umgesetzt und getestet. Das Projekt dokumentiert eigene Projekterfahrung; es stellt keine Berufserfahrung in einer produktiven Gebäudeanlage dar.

Profil des Autors

Ahmad Azroun verbindet einen Bachelor in Physik und eine Weiterbildung im Projektmanagement für erneuerbare Energien mit praktischer Weiterbildung in Gebäudeautomation. Dieses Projekt zeigt den Übergang von Messwerten und Energiegrundlagen zu Kommunikation, Steuerungslogik und Systemintegration.

Inhalt und Leseführung

Seiten	Thema
4-6	Projektziel, Abgrenzung und Systemarchitektur
7-9	Hardware, Netzwerk und MQTT
10-12	Modbus TCP, Hysterese und Betriebsarten
13-15	Ventilfreigabe, Überwachung und Alarmhistorie
16-17	Persistenz, Befehlsdauer und Energieabschätzung
18-22	Bilder: Dashboard, Hardware und ESP32-Testausgaben
23-26	Inbetriebnahme, Prüfplan und Troubleshooting
27-28	Portfolio-Einordnung, Grenzen und Quellen
29-41	Vollständige Firmware und Function-Code des aktuellen Exports

So lässt sich der Bericht lesen

• Für einen schnellen Überblick: Projektprofil, Architektur und kommentierte Dashboard-Bilder.

• Für eine technische Bewertung: Datenpunkte, Funktionsbeschreibung, Fehlerszenarien und Grenzen.

• Für die Reproduktion: Inbetriebnahme und Code-Anhang gemeinsam mit dem Repository verwenden.

> Sprache und Nachweise
> Der Bericht ist auf Deutsch verfasst; wichtige englische Fachbegriffe bleiben erhalten. Abbildungen stammen aus dem Projekt. Testergebnisse sind manuell beobachtete Ergebnisse des Laboraufbaus, keine unabhängige Zertifizierung.

Ziel und Entwicklungsschritt

Vom Mini-BMS zur realen Geräteanbindung

Das vorherige Mini-BMS-Projekt führte Setpoint, Istwert, Hysterese, AUTO/MANUAL, Command/Feedback, Alarme, Trends und Modbus TCP ein. Dieses Projekt erweitert die Grundlagen um ein reales Sensormodul, einen ESP32, MQTT und physische Ausgänge.

Technische Fragestellung

Wie wird ein Messwert aus einem Raum in ein übergeordnetes System übertragen, dort bewertet und als verständlicher, überprüfbarer Stellbefehl zurück an ein Gerät gesendet? Wie reagiert das System auf fehlende Daten, fehlende Rückmeldung und einen Neustart?

Neuer Baustein	Lernfortschritt
Realer DHT11	Messung statt ausschließlich Temperatur-Simulation
ESP32 und MQTT	Verteilung von Messung, Kommunikation und Ausgangssteuerung
Relais und Lüfter	Ein Softwarebefehl bewirkt einen realen Schaltvorgang
Ventil-LED	Getrennter zweiter Ausgang mit Freigabebedingungen
Alarmhistorie	Ereignisse statt nur momentaner Alarmzustände
Persistenz und Energie	Daten erhalten und aus Befehlsdauer eine Abschätzung berechnen

> Anwendungsbezug
> Die Demonstration verwendet typische BMS-Arbeitsweisen: Datenpunkte, Betriebsarten, Verriegelungen, Visualisierung, Diagnose und Dokumentation. Ein USB-Lüfter bildet dabei keinen vollständigen Raumkühlprozess ab.

Real, simuliert und geschätzt

Bestandteil	Art	Bedeutung
Temperatur / Feuchte	Real	Messwerte des DHT11
5-V-Lüfter	Real	Über ein Relais geschalteter Aktor
Lüfterrückmeldung	Simuliert	Inject/Modbus-Zustand; kein Drehzahlsensor
Kühlventil	Simuliert	Logik und LED, kein mechanisches Ventil
Automationsstation	Simuliert	Modbus-Server innerhalb von Node-RED
Laufzeit	Befehlsdauer	Zeit mit ON-Befehl, nicht nachgewiesener Lüfterlauf
Energie	Geschätzt	Befehlsdauer × angenommene 2 W

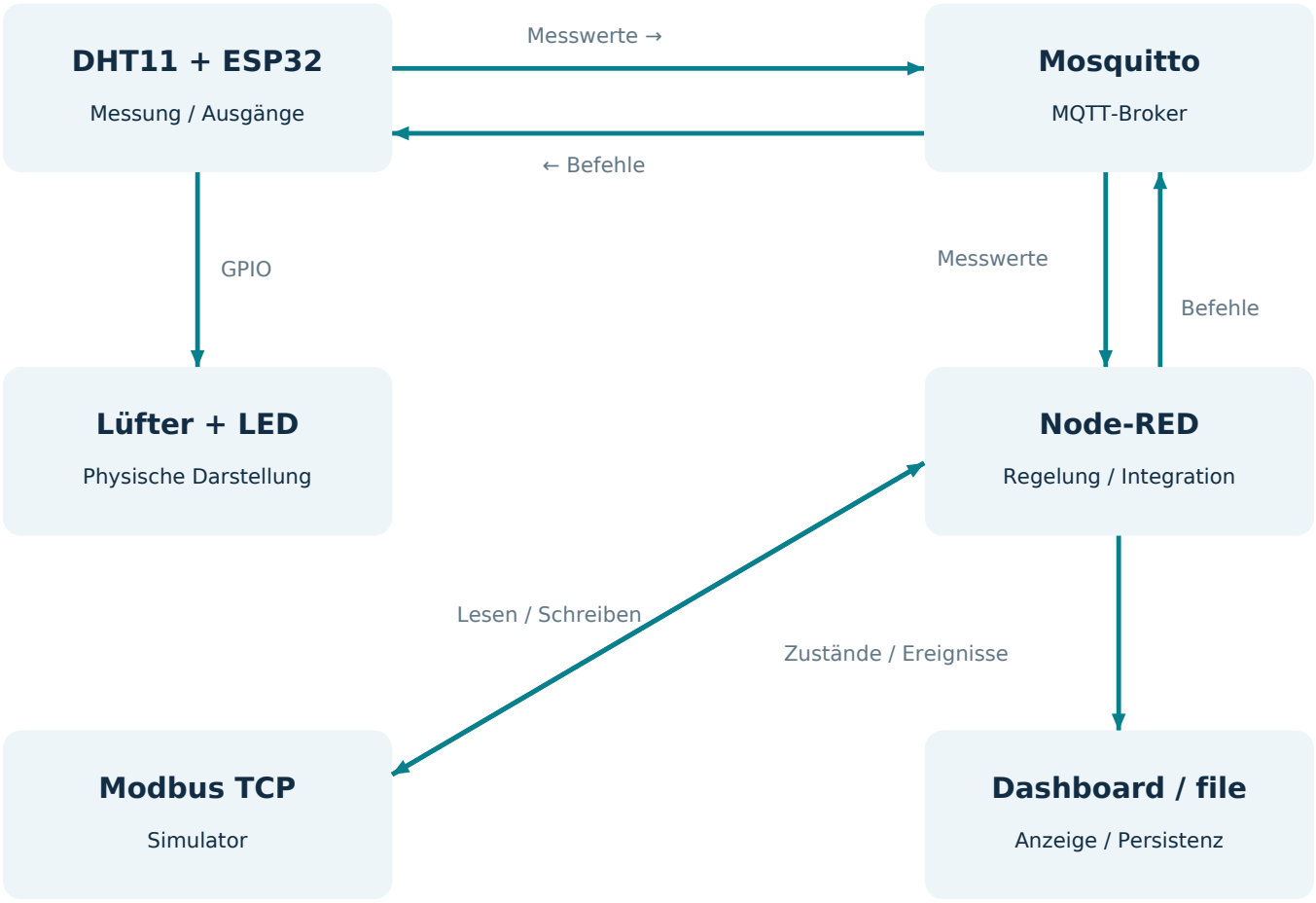
Warum diese Trennung wichtig ist

Ein Ausgang kann eingeschaltet sein, obwohl der angeschlossene Aktor nicht läuft. Ebenso kann eine simulierte Rückmeldung ON melden, obwohl keine reale Bewegung gemessen wurde. Das Projekt trennt deshalb den Befehl ausdrücklich von der Rückmeldung.

Die Energieanzeige ist kein Stromzähler. Sie zeigt, wie Zeitdaten und eine deklarierte Leistungsannahme verarbeitet werden. Eine spätere Erweiterung kann einen elektrischen Energiezähler oder reale Lüfterrückmeldung anbinden.

> Richtige Interpretation
> Eine leuchtende grüne LED bedeutet: Die Ventillogik hat den Ausgang freigegeben. Sie beweist weder einen Wasserstrom noch eine reale Ventilstellung. Eine rote Relaisanzeige beweist ebenfalls keine Drehzahl.

Systemarchitektur



Die Steuerungsentscheidung liegt in Node-RED. Der ESP32 erfasst Messwerte und setzt empfangene Befehle um. Mosquitto vermittelt Nachrichten; der Broker entscheidet nicht über den Kühlbedarf.

Modbus ergänzt einen simulierten Stationsbereich. Dashboard und Kontextspeicher sind zwei getrennte Ziele: Darstellung zeigt Zustände, Persistenz bewahrt ausgewählte Werte über Neustarts hinweg.

Hardware und Anschlüsse

Komponente	Aufgabe
Raspberry Pi	Host für Node-RED und Mosquitto
ESP32	MicroPython-Firmware, DHT11 und Ausgänge
DHT11	Temperatur und relative Luftfeuchtigkeit
2-Kanal-Relaismodul, 5 V	Genutzter Eingang IN2; Schalten der Lüfterversorgung
5-V-USB-Lüfter	Realer Demonstrationsaktor mit separater USB-Versorgung
LED mit Vorwiderstand	Darstellung der Kühlventilfreigabe

Signal	GPIO	Aktiver Zustand
DHT11-Datenleitung	4	Sensorprotokoll
Relais IN2	23	LOW = ON; HIGH = OFF
Ventil-LED	26	HIGH = ON; LOW = OFF

Beim Start setzt die Firmware das Relais auf HIGH und die LED auf LOW: beide Ausgänge sind ausgeschaltet. Die LED-Kathode ist mit GND verbunden; ein Vorwiderstand begrenzt den Strom.

> Anschlussbeschreibung
> Die GPIO-Zuordnung beschreibt den getesteten Aufbau. Versorgung, Eingangsschaltung und Pegel eines Relaismoduls müssen zum konkreten Modul passen. Die Versorgung des Lüfters wird vom Schaltkontakt geschaltet; sie ist von der GPIO-Befehlsleitung zu unterscheiden. Der Bericht enthält keinen universellen Schaltplan für unbekannte Relaismodule.

Netzwerk und Dienste

Knoten / Dienst	Wert im Labor
Raspberry Pi	192.168.178.38
ESP32	192.168.178.36
Windows-Laptop	192.168.178.25
Node-RED Editor	http://192.168.178.38:1880
Dashboard	http://192.168.178.38:1880/dashboard/page1
Mosquitto MQTT	TCP 1883
Modbus-Simulator	TCP 10502; Unit ID 1
Lokaler Node-RED-Client	127.0.0.1 für MQTT und Modbus

Loopback und LAN-Adresse

127.0.0.1 bezeichnet immer das Gerät, auf dem der Client läuft. Node-RED darf diese Adresse nutzen, weil Broker und Simulator auf demselben Pi laufen. Der ESP32 benötigt dagegen die LAN-Adresse des Pi; sein eigener Loopback führt nicht zum Pi.

Drei getrennte Diagnoseebenen

- IP-Erreichbarkeit: Ist das Gerät im Netzwerk erreichbar?
- TCP-Port: Akzeptiert der Dienst eine Verbindung auf dem Zielport?
- Anwendungsprotokoll: Funktionieren Authentifizierung, Topics und Datenpunkte?

> Beispiel aus der Fehlersuche
> Mosquitto war aktiv, lauschte anfangs aber nur auf 127.0.0.1 und ::1. Damit war eine lokale Verbindung möglich, der Zugriff vom ESP32 jedoch nicht. Ein laufender Dienst allein bestätigt keine LAN-Erreichbarkeit.

MQTT: Topics und Datenfluss

Topic	Richtung / Inhalt
bms/room1/temperature	ESP32 → Node-RED; numerischer Text in °C
bms/room1/humidity	ESP32 → Node-RED; numerischer Text in %
bms/room1/fan/command	Node-RED → ESP32; ON / OFF
bms/room1/valve/command	Node-RED → ESP32; ON / OFF

Die Firmware veröffentlicht Messwerte ungefähr alle zwei Sekunden mit QoS 0 und retain=False. Für die Befehlstopics wird in der aktuellen Lösung ebenfalls QoS 0 verwendet. Ein Publish-Aufruf ist bei QoS 0 kein quittierter Beweis, dass die Anwendung am Ziel den Wert verarbeitet hat.

Messwertverarbeitung

Der MQTT-Eingang wird in Node-RED in einen numerischen Temperaturwert umgewandelt. Ungültige Werte werden nicht als gültige Temperatur übernommen. Für Luftfeuchtigkeit gilt zusätzlich der Plausibilitätsbereich von 0 bis 100 %. Trends erhalten schlanke Nachrichten aus payload und topic.

Befehlsverarbeitung

Der ESP32 abonniert beide Befehlstopics. Nur ON und OFF sind gültige Nutzdaten. Die Fan-Nachricht wirkt auf GPIO23; die Valve-Nachricht auf GPIO26. Jeder gültige Befehl erneuert den eigenen Zeitstempel für die Befehlsüberwachung.

> Zugangsdaten
> Broker-Adresse: 192.168.178.38; Benutzer im Labor: bms_user. WiFi- und MQTT-Passwörter werden nur lokal eingetragen. Das veröffentlichte Firmware-Template enthält leere Passwortfelder.

Modbus TCP: Datenpunktmodell

Bereich / Funktion	Adresse	Bedeutung
Input Register / FC 4	0	Simulierte Temperatur; Lern- und Debugzweig
Holding Register / FC 3	0	Sollwert, Faktor 10
Coil / FC 1	0	Lüfterbefehl; Schreiben über FC 5
Discrete Input / FC 2	0	Simulierte Lüfterrückmeldung

Server: Port 10502; lokaler Client: 127.0.0.1:10502; Unit ID: 1. Die Adressen im Flow sind nullbasiert. Adresse 0 ist in den vier Bereichen jeweils ein anderer Datenpunkt.

Skalierung

Ein Integerwert 230 im Holding Register entspricht 23,0 °C. Ein simulierter Input-Register-Wert 246 entspricht 24,6 °C. Der Faktor 10 erhält eine Dezimalstelle innerhalb eines ganzzahligen Registers.

Aktuelle Regeltemperatur

Der reale Regelwert stammt in diesem Projekt vom DHT11 über MQTT. Der simulierte Modbus-Temperaturwert ist nur an einen Debugzweig angebunden. Ein erfolgreicher FC-4-Lesevorgang ersetzt deshalb nicht den MQTT-Sensorwert.

> Simulation der Rückmeldung
> Die Inject-Nodes Fan Feedback ON/OFF verändern den simulierten Discrete-Input-Bereich. Ein Modbus-Lesezweig übernimmt daraus fanFeedback. Das ist ein Testmodell für Integration, kein Rückkanal des physischen Lüfters.

Hysterese: Schalten mit Gedächtnis

Temperatur bei Sollwert 23 °C	Kühlbedarf
T ≥ 24 °C	ON
22 °C < T < 24 °C	Bisherigen Zustand beibehalten
T ≤ 22 °C	OFF

Die Funktion berechnet onThreshold = setpoint + 1 und offThreshold = setpoint - 1. coolingState wird im Flow-Kontext gespeichert. Ohne vorherigen Wert beginnt der Zustand mit false.

Beispielsequenz

Messwert	Vorher	Nachher	Erklärung
21 °C	OFF	OFF	Unter Ausschaltgrenze
24 °C	OFF	ON	Einschaltgrenze erreicht
23 °C	ON	ON	Innerhalb des Hysteresebands
22 °C	ON	OFF	Ausschaltgrenze erreicht

Das 2-K-Band reduziert wiederholtes Schalten nahe dem Sollwert. Es ist eine gewählte Regelstrategie des Laboraufbaus, kein allgemeiner Schutzabstand für jedes HVAC-Gerät.

> Grenzwerte genau lesen
> Die Meldungen zeigen OFF bereits bei 22 °C. Es ist nicht erforderlich, dass die Temperatur unter 22 °C fällt. Entsprechend wird ON bei 24 °C erreicht, nicht erst darüber.

AUTO und MANUAL

Betriebsart	Lüfterbefehl
AUTO	Folgt dem Kühlbedarf; bei dataTimeout = true auf OFF
MANUAL	Folgt manualFanCommand; Temperatur-Timeout überschreibt diesen Befehl nicht

Bedienung im aktuellen Projekt

Die Betriebsart und der manuelle Lüfterbefehl werden über Inject-Nodes im Node-RED-Editor gesetzt. Der aktuelle Flow bietet dafür keine Dashboard-Schaltflächen. Eine Änderung wirkt bei der nächsten Auswertung der Steuerlogik.

Initialwerte

- autoMode ist standardmäßig true.
- manualFanCommand ist standardmäßig false.
- fanFeedback ist standardmäßig false.
- Betriebsart und momentane Rückmeldung liegen im Arbeitsspeicher.

Befehl und physisches Gerät

fanCommand bezeichnet die Entscheidung von Node-RED. Ein MQTT-Vorbereitungszweig formatiert daraus ON oder OFF. Der ESP32 setzt den Relaisausgang entsprechend. Fan Feedback ist weiterhin ein separater simulierter Wert.

> Wichtig beim Testen
> MANUAL ON kann den Lüfter ohne aktuellen Kühlbedarf einschalten. Die Ventil-LED benötigt zusätzlich Kühlbedarf, simulierte Rückmeldung und gültige Temperaturdaten. Deshalb bedeutet MANUAL ON allein nicht automatisch Ventil OFFEN.

Ventilfreigabe und LED

Die Function Kühlventil Command gibt den Ausgang nur frei, wenn vier Bedingungen gleichzeitig erfüllt sind. Die resultierende Boolesche Entscheidung wird als eigener MQTT-Befehl zum ESP32 übertragen.

Bedingung	Erforderlich
dataTimeout	false
coolingDemand	true
fanCommand	true
fanFeedback	true

```
msg.valveCommand =
    !dataTimeout &&
    coolingDemand &&
    fanCommand &&
    fanFeedback;
```

Statusdarstellung

Bei true wird valveStatus auf OFFEN gesetzt, bei false auf GESCHLOSSEN. Die grüne LED leuchtet bei ON auf GPIO26. Die Bezeichnung stellt die Freigabe dar; es wird kein mechanischer Ventilhub gemessen.

Typische Prüffälle

• Kühlbedarf ON, Befehl ON, Feedback OFF: Ventil bleibt geschlossen.

• Feedback ON bei unverändertem Bedarf: Ventil wird freigegeben.

• Temperatur fällt auf 22 °C: Bedarf OFF, Lüfterbefehl OFF, Ventil geschlossen.

• Temperaturdaten-Timeout: Ventilfreigabe wird zurückgenommen.

Überwachung und Wiederverbindung

Überwachung	Zeit / Verhalten
Temperaturdaten in Node-RED	5 s ohne gültige Temperatur: Timeout-Nachricht
Fehlende simulierte Fan-Rückmeldung	Alarm nach mindestens 5 s bei nächster Steuerungsnachricht
Befehl auf dem ESP32	10 s ohne erneuten gültigen Befehl: entsprechender Ausgang OFF
WiFi-Verbindungsversuch	20 s Wartezeit je Versuch
Wiederholungsversuch	Pause 5 s; anschließend erneut verbinden

Bei erkanntem WiFi- oder MQTT-Verlust schaltet die Firmware beide Ausgänge aus und schließt die Verbindung. Nach erneuter WiFi-Verbindung wird MQTT neu verbunden und beide Befehlstopics werden erneut abonniert.

Nachrichtenabhängiger Fan-Alarm

Ein ON-Befehl bei Feedback OFF startet feedbackMissingSince. Vor Ablauf der Wartezeit wird fanStartPending gesetzt. Ab fünf Sekunden führt die nächste Auswertung zum Alarm. Befehl OFF oder Feedback ON setzt die Überwachung zurück.

> Zeitverhalten und Grenzen
> Die Fan-Alarmfunktion wird durch eingehende Nachrichten ausgewertet. Die ESP32-Zeitüberwachung läuft in der Programmschleife. Blockierende Netzwerkoperationen können Abschaltungen verzögern. Diese Zeiten sind keine garantierten Echtzeit-Sicherheitsgrenzen.

Alarmhistorie: letzte drei Ereignisse

Feld	Bedeutung
raisedAt	Zeitpunkt der erkannten neuen Alarmbedingung
clearedAt	Zeitpunkt des erkannten Endes; bei aktivem Alarm leer
status	AKTIV oder GEGANGEN
text	Lüfter: Keine Rückmeldung bei Einschaltbefehl

Bei einem neuen Alarm wird genau ein Datensatz vorne eingefügt. Weitere Nachrichten mit demselben aktiven Zustand erzeugen keinen neuen Eintrag. Beim Alarmende wird derselbe Datensatz aktualisiert. history.slice(0, 3) begrenzt die Liste auf die letzten drei Ereignisse.

Zeitstempel und Bedeutung

Die Darstellung verwendet de-DE und Europe/Berlin. GEGANGEN bedeutet, dass die Alarmbedingung bei der Auswertung nicht mehr vorliegt. Es bedeutet keine Quittierung, keine Reparaturbestätigung und keinen tatsächlichen Drehzahlnachweis.

Darstellung im Dashboard

Die Tabelle zeigt Gekommen, Gegangen, Status und Meldung. Items per page und die Pfeile gehören zur Tabellen-Navigation. Sie sind keine weiteren Alarme und kein Anlagenzustand.

> Neustart
> Die Historie wird im benannten file-Kontext gespeichert. Nach einer Unterbrechung bezeichnet eine neu erfasste Ende-Zeit den ersten beobachteten Normalzustand nach Wiederaufnahme; die Anlage rekonstruiert keine Ereigniszeit innerhalb eines unbeobachteten Ausfalls.

Persistenz und Befehlsdauer

```
contextStorage: {
    default: "memoryOnly",
    memoryOnly: { module: "memory" },
    file: { module: "localfilesystem" }
},
```

Die Konfiguration gehört innerhalb des exportierten Einstellungsobjekts in settings.js und außerhalb von Kommentaren. Eine vorhandene contextStorage-Eigenschaft wird ersetzt, nicht dupliziert.

Wert	Speicher
totalMs	file: akkumulierte ON-Befehlsdauer
fanAlarmHistory	file: letzte drei Alarmereignisse
lastUpdate / previousCommand	Standard: memoryOnly
Betriebsart / Kühlzustand / Rückmeldung	Standard: memoryOnly

Der Zähler addiert das vorangegangene Intervall, wenn der vorherige Befehl ON war. Intervalle über fünf Sekunden werden ausgelassen, um längere unbeobachtete Lücken nicht mitzuzählen. Sekunden werden abgerundet; Stunden verwenden die vollständigen Millisekunden.

> Praktischer Nachweis
> Vor einem geordneten Node-RED-Neustart waren 18 s gespeichert. Nach dem Neustart blieb die Anzeige bei 18 s. Auch die Alarmhistorie blieb erhalten. Für den Test wurde die Schreibfrist des Dateispeichers berücksichtigt; bei Standardkonfiguration etwa 30 s, mit mindestens 35 s Wartezeit nach der letzten Änderung.

Energieabschätzung

Annahme bewusst offenlegen

Für den Lernaufbau wird eine konstante Leistung von 2 W angenommen. Dieser Wert wurde weder elektrisch gemessen noch aus einem Datenblatt übernommen. Die Berechnung nutzt die gespeicherte ON-Befehlsdauer.

```
E_geschaetzt [Wh] = Dauer_ON [h] * 2 W
Dauer_ON [h] = totalMs / 3600000
2990 / 3600 * 2 = 1.661111... Wh
```

ON-Befehlsdauer	Abschätzung bei 2 W
18 s	0,010 Wh
1800 s	1,000 Wh
3600 s	2,000 Wh

Die Dashboard-Ausgabe rundet die Energie auf drei Nachkommastellen. Bei 2990 angezeigten Sekunden sind etwa 1,661 Wh zu erwarten; 1,662 Wh kann durch die Verwendung der vollständigen Millisekunden entstehen. Die Sekundenanzeige wird zuvor abgerundet.

Verhalten bei OFF

Nach Verarbeitung des letzten ON-Intervalls bleiben Dauer und Energie konstant. Die Energie wird aus der persistenten Zeit neu berechnet und benötigt keinen zweiten persistenten Zähler.

> Energy Monitoring als nächster Schritt
> Für eine echte Verbrauchsmessung wären elektrische Messwerte oder ein Energiezähler erforderlich. In diesem Projekt ist die Anzeige ausdrücklich als Energie geschätzt (2 W angenommen) beschriftet.

Dashboard: Kühlbetrieb

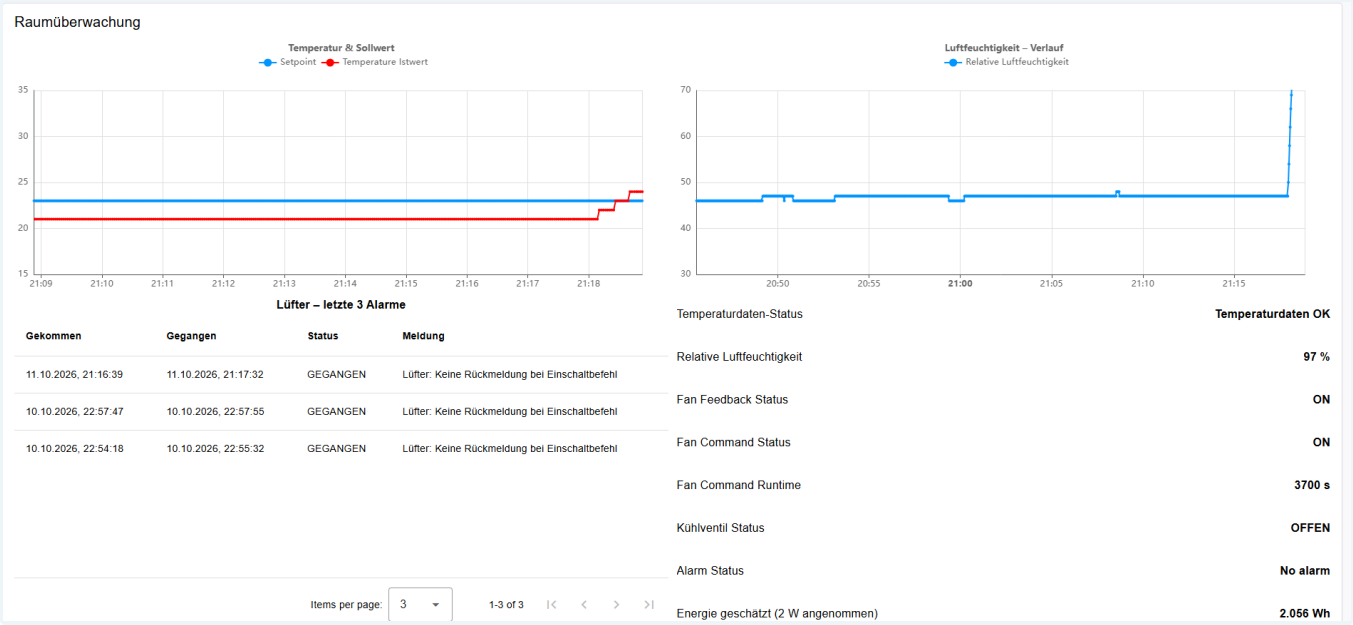


Abbildung 1. Dashboard im Kühlbetrieb: aktueller Messwert, Trends, Freigaben und gespeicherte Alarmereignisse.

Sichtbarer Zustand	Interpretation
Temperatur 24 °C	Einschaltgrenze bei Sollwert 23 °C erreicht
Fan Command ON / Feedback ON	Befehl aktiv und simulierte Rückmeldung vorhanden
Kühlventil OFFEN	Alle Freigabebedingungen erfüllt
No alarm	Aktuell keine fehlende simulierte Fan-Rückmeldung
Laufzeit / Energie	Gespeicherte Befehlsdauer; rechnerische Abschätzung

Die Temperaturkurve wird zusammen mit dem Sollwert dargestellt. Der Feuchtetrend zeigt einen Anstieg während des Tests. Die ältere Alarmhistorie bleibt sichtbar, obwohl aktuell kein Alarm aktiv ist.

> Anzeigefenster
> Der Feuchtetrend im Export besitzt einen festen Bereich von 30 bis 70 %. Der hohe Testwert liegt darüber und kann abgeschnitten erscheinen. Der numerische Messwert und das Trendfenster sind daher gemeinsam zu lesen.

Dashboard: aktiver Alarm

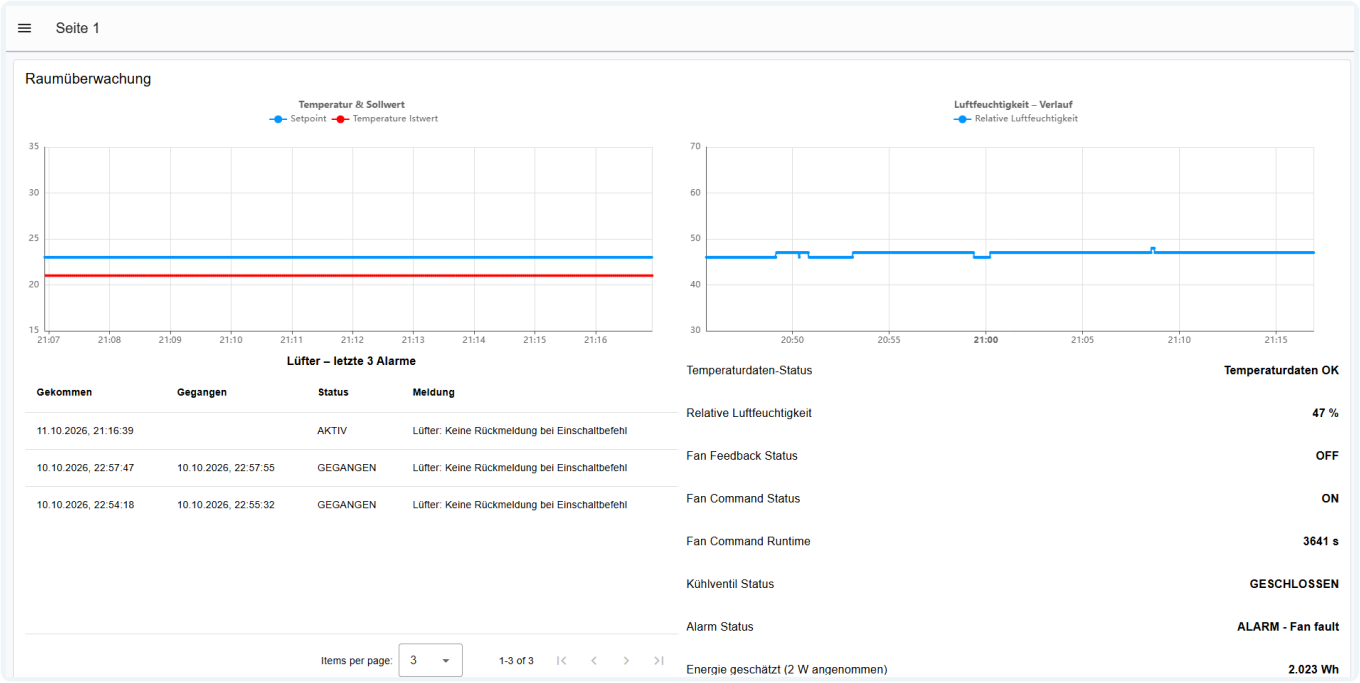


Abbildung 2. Testzustand: Fan Command ON, simuliertes Feedback OFF und aktiver Lüfteralarm.

Was diese Aufnahme nachweist

Der Alarmtest zeigt die Trennung von Command und Feedback. Ein eingeschalteter Befehl ohne simulierte Rückmeldung führt nach der Wartezeit zu einem Alarm. Die aktuelle Zeile in der Historie steht auf AKTIV; ihr Ende ist noch leer.

Die Ventilfreigabe bleibt GESCHLOSSEN, weil fanFeedback fehlt. Die Momentaufnahme mit einem ON-Befehl bei niedriger Temperatur allein erlaubt keine sichere Aussage über die Betriebsart; MANUAL kann unabhängig von der Hysterese einen ON-Befehl erzeugen.

> Alarm ist ein Ereignis
> Der Alarmzustand wird mit jeder Auswertung weitergegeben. Die Historie legt jedoch nicht alle zwei Sekunden einen neuen Alarm an. Nur der Wechsel in einen neuen aktiven Zustand erzeugt einen Eintrag.

Dashboard: Befehl OFF

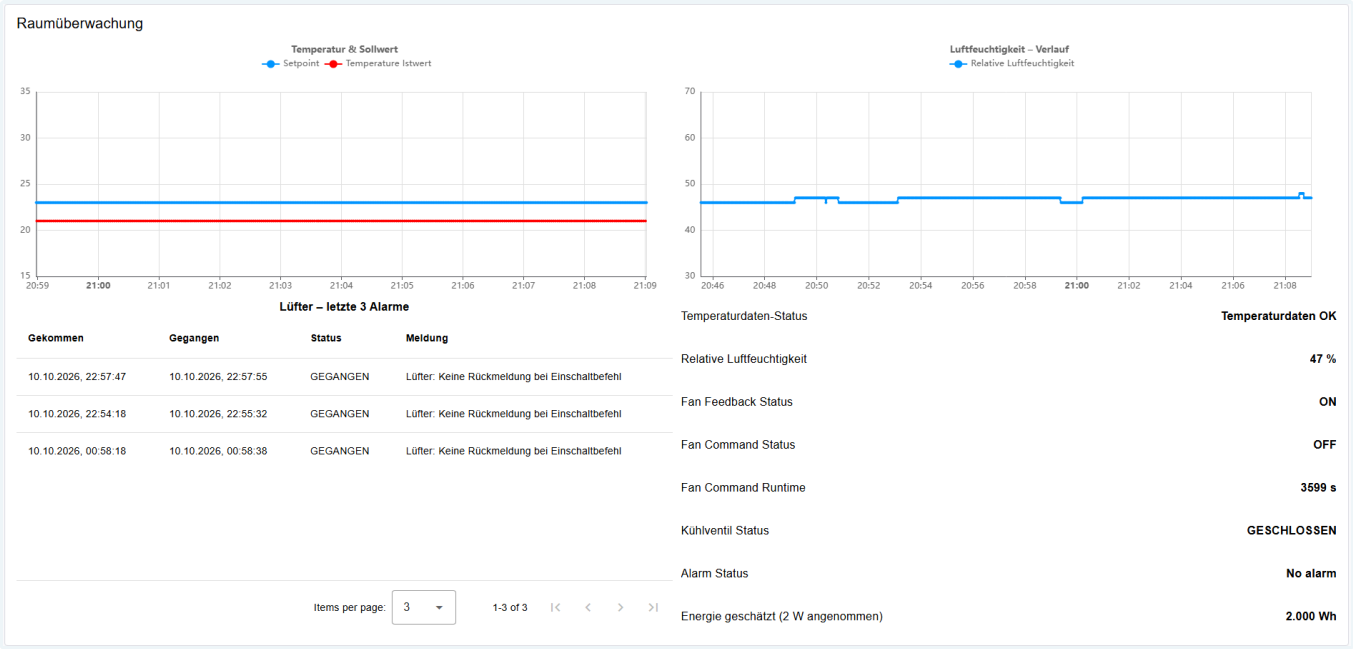


Abbildung 3. Fan Command OFF, Kühlventil GESCHLOSSEN und aktuell kein Lüfteralarm.

Befehl und Rückmeldung getrennt lesen

Die Temperatur liegt in dieser Aufnahme bei 21 °C. Fan Command ist OFF und das Ventil ist geschlossen. Die simulierte Rückmeldung steht weiterhin auf ON. Dieser Zustand wurde im Test nicht automatisch an den realen Lüfter gekoppelt.

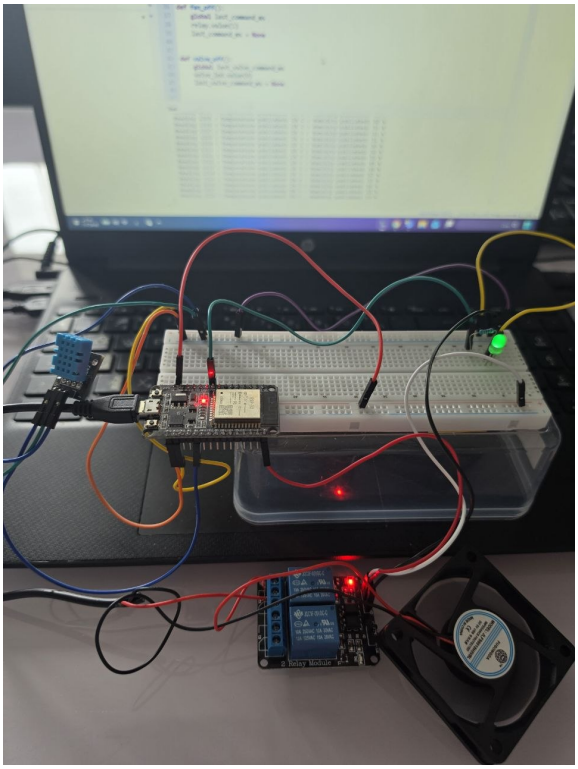
Die aktuelle Alarmfunktion überwacht ON ohne Feedback. Sie erzeugt keinen separaten Alarm für OFF bei Feedback ON. Eine spätere Erweiterung könnte auch einen Nachlauf oder einen unerwarteten Betrieb überwachen.

Gespeicherte Werte

Befehlsdauer und Energie bleiben nach dem letzten verarbeiteten ON-Intervall erhalten. Die Historie zeigt vergangene Ereignisse mit Status GEGANGEN. Die Anzeige bleibt dadurch auch im ausgeschalteten Zustand für Diagnose und Dokumentation nützlich.

> Kein Widerspruch im Bild
> Feedback ON bei Command OFF ist hier eine bewusste Folge der unabhängigen Simulation. Es ist kein Beweis, dass der physische Lüfter trotz OFF läuft.

Hardware: ON und OFF



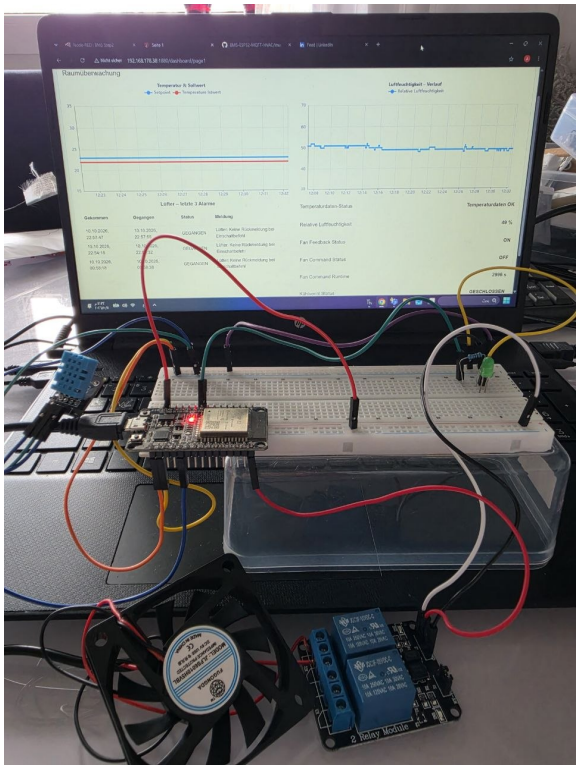


Abbildung 4 (links): Ventil-LED eingeschaltet und Relaisanzeige aktiv. Abbildung 5 (rechts): Ventil-LED ausgeschaltet; der Aufbau bleibt versorgt.

Bauteile im Versuchsaufbau

Auf dem Breadboard befinden sich ESP32, Sensoranschluss und die grüne Ventil-LED. Das Relaismodul schaltet die separate 5-V-USB-Versorgung des Lüfters. Der Laptop zeigt die Softwareebene des Experiments.

Bilder richtig bewerten

Eine statische Aufnahme dokumentiert die Verdrahtung und sichtbare Anzeigen. Sie kann die Drehzahl oder einen tatsächlichen Lüfterstillstand nicht messen. Der reale Betrieb wurde im praktischen Test beobachtet und ist ergänzend im Hardwarevideo demonstriert.

> Grüne und rote LEDs
> Die grüne LED repräsentiert das Kühlventil. Eine rote ESP32-Versorgungsanzeige kann auch bei ausgeschaltetem Lüfter und Ventil leuchten. OFF bedeutet ausgeschalteter Aktorbefehl, nicht zwingend vollständig spannungsfreier Aufbau.

ESP32: Testausgaben 24 / 22 °C

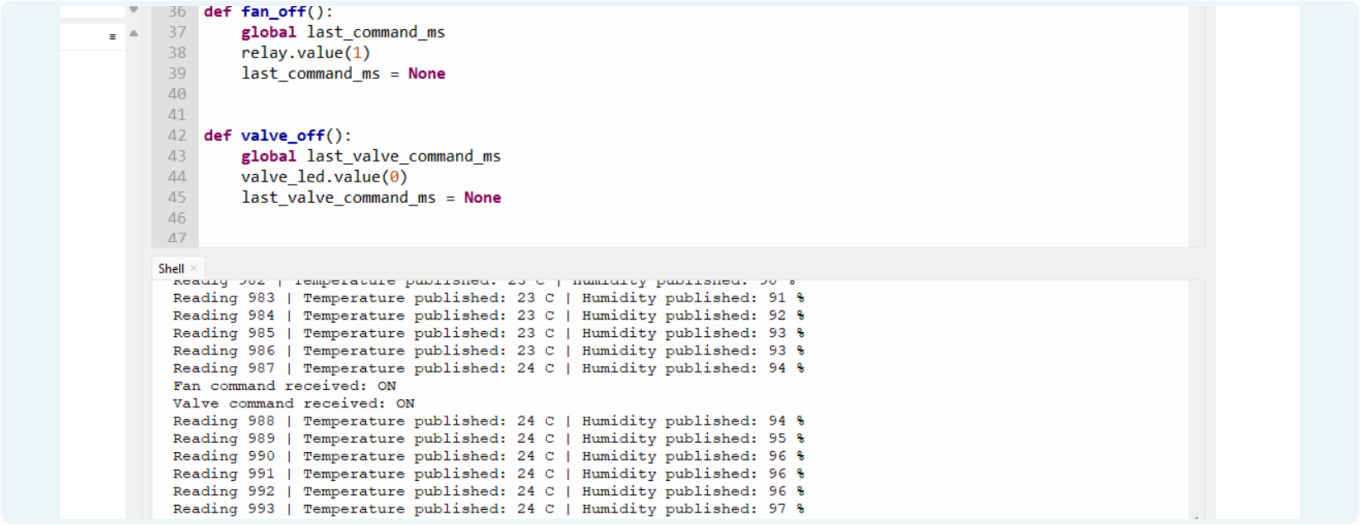


Abbildung 6. Thonny/ESP32: Messwerte und ON-Befehle beim Erreichen von 24 °C.

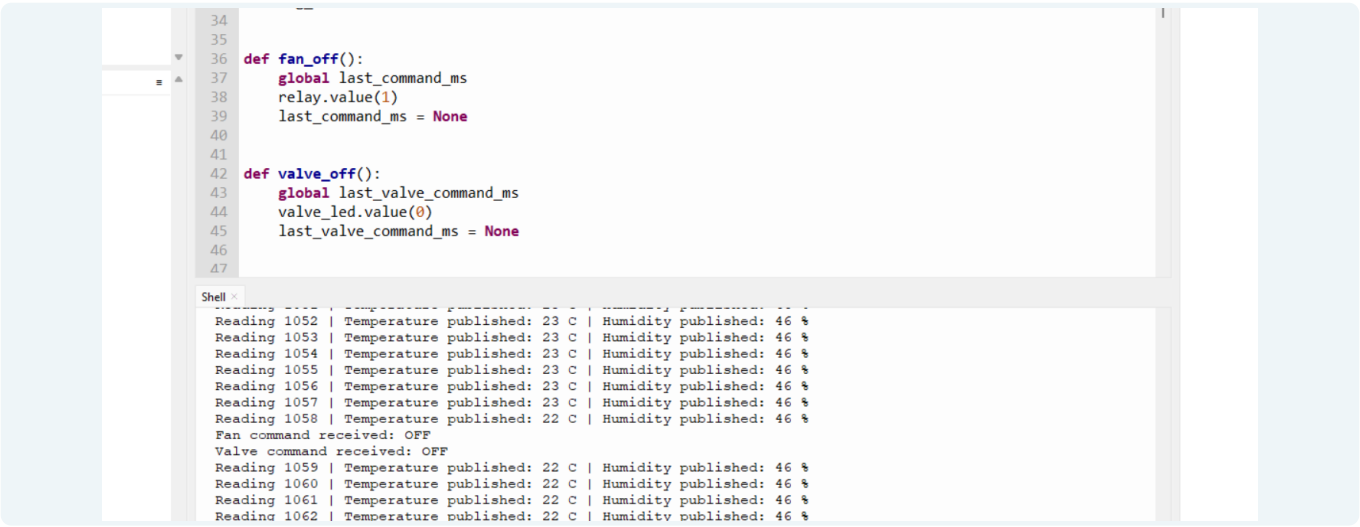


Abbildung 7. Thonny/ESP32: Fan und Valve OFF beim Erreichen von 22 °C.

Die Ausschnitte vergrößern die relevanten Shell-Bereiche der vollständigen Projektbilder. Temperature published dokumentiert den Publish-Aufruf der Firmware. Fan command received und Valve command received zeigen, dass der Callback die jeweiligen Befehle empfangen hat.

> Schaltfolge
> Die dokumentierten ON- und OFF-Meldungen passen zur Hysterese mit 24 °C und 22 °C. Die Shell ersetzt keinen echten Rückmeldekontakt; sie dokumentiert Sensorabfrage, Kommunikation und Ausgangsentscheidung.

Node-RED: Flow und Inbetriebnahme

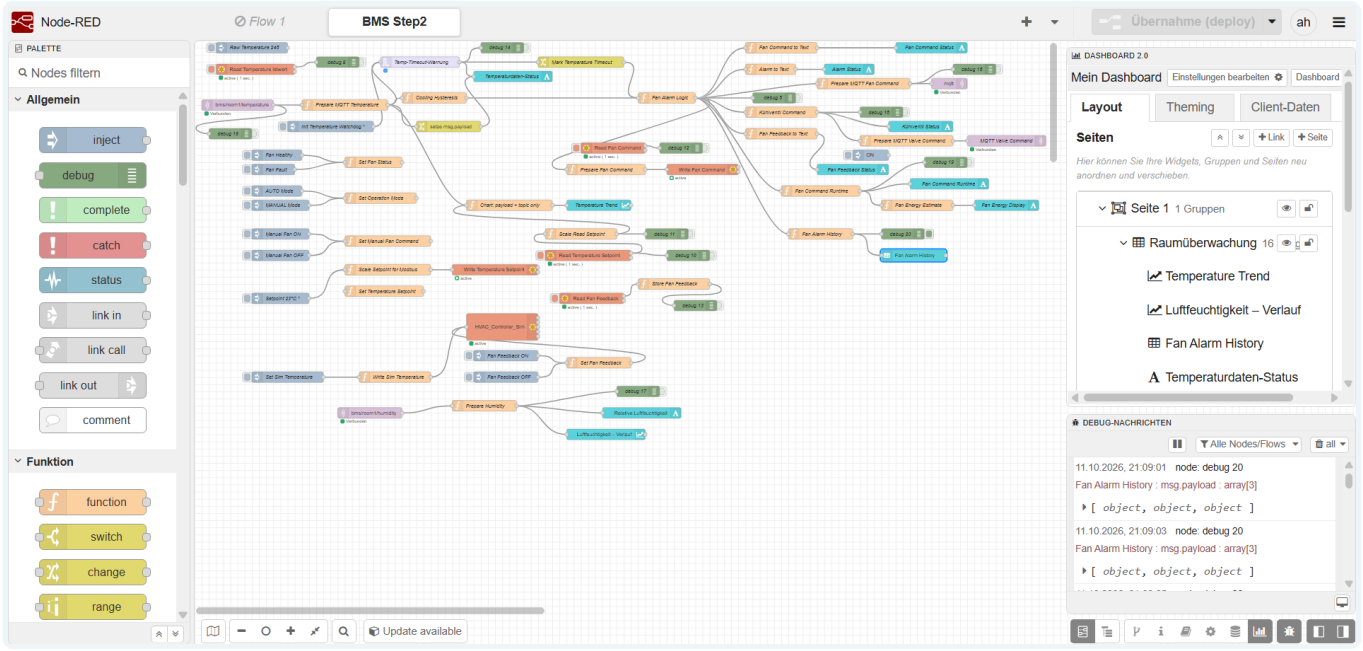


Abbildung 8. Vollständige Editor-Aufnahme des veröffentlichten Tabs BMS Step2.

Der aktuelle Export enthält 83 Nodes einschließlich Konfigurationsknoten. Die Hauptzweige verbinden MQTT-Messwerte, Hysterese, Fan-Alarm-Logik, Ventilfreigabe und Befehlstopics. Weitere Zweige führen zu Modbus, Dashboard, Alarmhistorie und Energieabschätzung.

Einrichtung in der richtigen Reihenfolge

• Node-RED, Mosquitto und Module bereitstellen. Dashboard: 1.32.0; Modbus: 5.60.2 im dokumentierten Aufbau.

• file-Kontext in der aktiven settings.js einrichten, Syntax prüfen und Node-RED neu starten.

• Flow importieren; MQTT-Zugangsdaten und lokale Modbus-Verbindungen prüfen; Deploy ausführen.

• Firmware lokal mit Zugangsdaten ergänzen und als main.py auf den ESP32 speichern.

• Messwerte prüfen, anschließend Freigaben und Fehlerszenarien testen.

Dienststart und Reproduktion

Raspberry Pi

```
sudo apt install mosquitto mosquitto-clients
sudo systemctl enable --now mosquitto
sudo systemctl enable nodered.service
sudo systemctl start nodered
systemctl is-active nodered mosquitto
```

Mosquitto muss für den ESP32 über das LAN erreichbar sein. Listener, Benutzer und Passwortdatei sind lokal einzurichten; die individuelle Broker-Konfiguration wird im Repository nicht veröffentlicht.

Konfiguration prüfen

```
node --check /home/ahmad/.node-red/settings.js
sudo systemctl restart nodered
journalctl -u nodered -n 40 --no-pager
```

Im Startlog müssen memoryOnly und file / localfilesystem erkennbar sein. Eine erfolgreiche Syntaxprüfung allein zeigt nicht, ob eine eingefügte Konfiguration versehentlich innerhalb eines Kommentars liegt.

Exportumfang

Flow und Firmware enthalten keine historischen Kontextwerte oder Brokerpasswörter. Ein neuer Aufbau startet ohne die gespeicherten Zeit- und Alarmwerte des ursprünglichen Pi. Andere Flows können dieselbe Dashboard-Gruppe oder Ports verwenden; vor Änderungen ist ihre Herkunft zu prüfen.

Prüfplan und beobachtete Ergebnisse

Prüfung	Beobachtung im Labor
AUTO: Temperatur auf 24 °C	Fan ON; bei Feedback ON auch Ventil-LED ON
Temperatur zurück auf 22 °C	Fan OFF und Ventil geschlossen
Command ON / Feedback OFF	Verzögerter Fan-Alarm; neue aktive Historienzeile
Feedback ON oder Command OFF	Alarm endet; bestehende Zeile erhält Ende-Zeit
Mehr als drei Alarmereignisse	Liste bleibt auf die letzten drei begrenzt
MQTT-Unterbrechung	Wiederholungsversuche; danach erneute Übertragung
WiFi-Unterbrechung	WiFi und MQTT verbinden sich erneut
Fehlende Temperaturdaten	Datenwarnung; AUTO-Fan-Befehl OFF
Node-RED-Neustart	Befehlsdauer und Alarmhistorie bleiben erhalten
Befehl OFF	Zeit und Energie nach letztem ON-Intervall konstant

Die Logausgaben belegen unter anderem eine erfolgreiche Wiederaufnahme nach MQTT- und WiFi-Unterbrechungen. Die Hysterese wurde mit realen Sensorwerten geprüft. Ein Persistenztest zeigte den unveränderten Zählerstand von 18 s nach Neustart.

> Nachweisumfang
> Diese Ergebnisse stammen aus manuellen Versuchen am konkreten Laboraufbau. Es wurde keine allgemeine Verfügbarkeit, garantierte Abschaltzeit oder reproduzierbare HVAC-Raumkühlleistung vermessen.

Troubleshooting und Ursachenanalyse

Symptom	Prüfung / typische Ursache
MQTT nicht erreichbar	Brokerdienst, TCP 1883, LAN-Listener, IP und Authentifizierung prüfen
Node-RED lädt nicht	Dienststatus prüfen; in einem Test war nodered inactive (dead)
DHT11: ETIMEDOUT	GPIO4, Versorgung und Steckverbindungen prüfen; MQTT kann trotzdem funktionieren
COM3 nicht gefunden	Aktuellen seriellen Port und USB-Verbindung in Thonny prüfen
Zähler nach Neustart null	Aktive settings.js und benannten file-Speicher prüfen
Doppelte Dashboard-Elemente	Herkunft aus weiteren Tabs und gemeinsam genutzte Gruppen prüfen
LED bleibt bei MANUAL ON aus	Kühlbedarf, Feedback und dataTimeout gemeinsam prüfen
Fan endet nach etwa 10 s	Befehle müssen rechtzeitig wiederholt werden; Timeout prüfen

Befehle für die Diagnose

```
systemctl status mosquitto --no-pager
sudo ss -ltnp 'sport = :1883'
systemctl status nodered --no-pager
hostname -I
```

Eine Schicht nach der anderen prüfen: Netzwerk, Dienst, Authentifizierung, Nachricht, Steuerungszustand, Ausgang. Dadurch lassen sich Kommunikationsfehler von Sensor- und Logikfehlern trennen.

Portfolio und nächster Entwicklungsschritt

Was dieses Projekt demonstriert

- Datenpunkte und Schnittstellen strukturiert definieren.
- Messung, Kommunikation und Steuerungsentscheidung trennen.
- Betriebsarten, Hysterese und Freigabebedingungen implementieren.
- Alarme als Zustände und Ereignisse unterscheiden.
- Systemfehler aus Logs und beobachtetem Verhalten eingrenzen.
- Persistenz, Visualisierung und Energieannahmen nachvollziehbar dokumentieren.

Formulierung für ein Fachgespräch

> Eigene Projekterfahrung
> Ich habe einen Laboraufbau mit ESP32, DHT11, MQTT und Node-RED entwickelt und dokumentiert. Darin habe ich Hysterese, AUTO/MANUAL, Command/Feedback, Alarmhistorie, persistente Zähler und eine deklarierte Energieabschätzung umgesetzt. Die Lüfterrückmeldung ist simuliert; der Lüfter selbst wird real über ein Relais geschaltet.

Nächste fachliche Schritte

Eine Multi-Zone-Demonstration kann die Datenpunktstruktur, getrennte Zonenregler und den Energievergleich erweitern. BACnet/IP ergänzt anschließend Geräte, Objekte, Eigenschaften und Systemintegration. Reale Rückmeldung und ein Energiezähler würden die bisherigen Simulationen und Annahmen durch zusätzliche Messdaten ergänzen.

Der Bericht belegt übertragbare Grundlagen für technische Projektarbeit, Integration und Visualisierung. Er ersetzt keine Einweisung in eine herstellerspezifische Automationsstation und macht keine Aussage über eine konkrete Einstellungsentscheidung.

Quellen, Dateien und Projektgrenzen

Primäre Projektquelle

github.com/ahmad-energy-iot/BMS-ESP32-MQTT-HVAC

Datei	Inhalt
README.md	Projektübersicht und Medien
docs/PROJECT_DOCUMENTATION.md	Technische Projektdokumentation
flows/bms-step2-flow.json	Vollständiger Node-RED-Export
firmware/esp32_bms_step2.py	MicroPython ohne veröffentlichte Passwörter
config/context-storage.example.js	Beispiel für memoryOnly und file
images/	Acht Projektbilder; im Bericht vollständig erläutert
videos/Add hardware demonstration video.mp4	Hardwarevideo, ergänzender Bewegungsnachweis

Hardware-Demonstration auf GitHub öffnen

Stand und Abgrenzung

Stand: 11.10.2026. Ein Raum; simulierte Rückmeldung; LED statt physischem Ventil; Energieabschätzung statt elektrischer Messung. Keine BACnet/IP-Implementierung in dieser Version. Kein Nachweis einer produktiven Anlage oder zertifizierten Sicherheitssteuerung.

Der folgende Anhang enthält die aktuelle Firmware und alle 24 Function-Inhalte des veröffentlichten Flows. Der JSON-Export bleibt für Verdrahtung, Konfigurationsknoten und Dashboard-Einstellungen maßgeblich.

Anhang A: ESP32-Firmware

Vollständige Firmware aus firmware/esp32_bms_step2.py. Zugangsdaten bleiben leer und werden lokal ergänzt. Umgebrochene Zeilen dienen nur der Lesbarkeit des Berichts.

```
001 from machine import Pin
002 from dht import DHT11
003 from umqtt.simple import MQTTClient
004 import network
005 import time
006
007 # Copy your existing connection settings here. Do not share passwords.
008 SSID = ""
009 PASSWORD = ""
010 MQTT_BROKER = "192.168.178.38"
011 MQTT_USER = "bms_user"
012 MQTT_PASSWORD = ""
013
014 MQTT_TOPIC = b"bms/room1/temperature"
015 HUMIDITY_TOPIC = b"bms/room1/humidity"
016 FAN_TOPIC = b"bms/room1/fan/command"
017 VALVE_TOPIC = b"bms/room1/valve/command"
018 READ_INTERVAL_MS = 2000
019 COMMAND_TIMEOUT_MS = 10000
020 RETRY_INTERVAL = 5
021 WIFI_TIMEOUT_MS = 20000
022
023 # Fan relay: IN2 -> GPIO23, active LOW.
024 relay = Pin(23, Pin.OUT, value=1)
025 # GPIO26 -> resistor -> LED long leg; short leg -> GND.
026 valve_led = Pin(26, Pin.OUT, value=0)
027 sensor = DHT11(Pin(4))
028 wlan = network.WLAN(network.WLAN.IF_STA)
029 wlan.active(True)
030 client = None
031 last_command_ms = None
032 last_valve_command_ms = None
033 reading_number = 0
034
035
036 def fan_off():
037     global last_command_ms
038     relay.value(1)
039     last_command_ms = None
040
041
042 def valve_off():
043     global last_valve_command_ms
044     valve_led.value(0)
045     last_valve_command_ms = None
046
047
048 def on_message(topic, message):
```

Anhang A: ESP32-Firmware

Vollständige Firmware aus firmware/esp32_bms_step2.py. Zugangsdaten bleiben leer und werden lokal ergänzt. Umgebrochene Zeilen dienen nur der Lesbarkeit des Berichts.

```
049     global last_command_ms, last_valve_command_ms
050     if topic == FAN_TOPIC:
051         if message not in (b"ON", b"OFF"):
052             fan_off()
053             print("Invalid fan command: fan OFF")
054             return
055         last_command_ms = time.ticks_ms()
056         desired = 0 if message == b"ON" else 1
057         changed = relay.value() != desired
058         relay.value(desired)
059         if changed:
060             print("Fan command received:", message.decode())
061     elif topic == VALVE_TOPIC:
062         if message not in (b"ON", b"OFF"):
063             valve_off()
064             print("Invalid valve command: LED OFF")
065             return
066         last_valve_command_ms = time.ticks_ms()
067         desired = 1 if message == b"ON" else 0
068         changed = valve_led.value() != desired
069         valve_led.value(desired)
070         if changed:
071             print("Valve command received:", message.decode())
072
073
074 def close_mqtt():
075     global client
076     fan_off()
077     valve_off()
078     if client is not None:
079         try:
080             client.sock.close()
081         except Exception:
082             pass
083     client = None
084
085
086 def retry_pause():
087     fan_off()
088     valve_off()
089     for _ in range(RETRY_INTERVAL * 10):
090         time.sleep_ms(100)
091
092
093 def connect_wifi():
094     if wlan.isconnected():
095         return True
096     print("Connecting to WiFi...")
```

Anhang A: ESP32-Firmware

Vollständige Firmware aus firmware/esp32_bms_step2.py. Zugangsdaten bleiben leer und werden lokal ergänzt. Umgebrochene Zeilen dienen nur der Lesbarkeit des Berichts.

```
097     try:
098         wlan.active(True)
099         wlan.disconnect()
100         wlan.connect(SSID, PASSWORD)
101         start = time.ticks_ms()
102         while not wlan.isconnected():
103             if time.ticks_diff(time.ticks_ms(), start) >= WIFI_TIMEOUT_MS:
104                 wlan.disconnect()
105                 print("WiFi connection timed out")
106                 return False
107             time.sleep_ms(100)
108         print("WiFi connected | ESP32 IP:", wlan.ifconfig()[0])
109         return True
110     except OSError as error:
111         print("WiFi ERROR:", error)
112         return False
113 
114 
115 def connect_mqtt():
116     global client
117     close_mqtt()
118     try:
119         client = MQTTClient(
120             client_id=b"bms-step2-esp32",
121             server=MQTT_BROKER,
122             port=1883,
123             user=MQTT_USER.encode(),
124             password=MQTT_PASSWORD.encode(),
125             keepalive=30
126         )
127         client.set_callback(on_message)
128         client.connect()
129         client.subscribe(FAN_TOPIC)
130         client.subscribe(VALVE_TOPIC)
131         print("MQTT connected | subscribed:", FAN_TOPIC.decode())
132         print("Subscribed:", VALVE_TOPIC.decode())
133         return True
134     except Exception as error:
135         print("MQTT connection ERROR:", repr(error))
136         close_mqtt()
137         return False
138 
139 
140 try:
141     if not SSID or not PASSWORD or not MQTT_BROKER or not MQTT_PASSWORD:
142         raise ValueError("Fill your existing WiFi and MQTT settings first")
143     last_read_ms = time.ticks_ms()
144     while True:
```

Anhang A: ESP32-Firmware

Vollständige Firmware aus firmware/esp32_bms_step2.py. Zugangsdaten bleiben leer und werden lokal ergänzt. Umgebrochene Zeilen dienen nur der Lesbarkeit des Berichts.

```
145         if not wlan.isconnected():
146             close_mqtt()
147             if not connect_wifi():
148                 retry_pause()
149                 continue
150         if client is None:
151             if not connect_mqtt():
152                 retry_pause()
153                 continue
154         try:
155             if (last_command_ms is not None and
156                     time.ticks_diff(time.ticks_ms(), last_command_ms)
157                     >= COMMAND_TIMEOUT_MS):
158                 fan_off()
159                 print("Fan OFF: command timeout")
160             if (last_valve_command_ms is not None and
161                     time.ticks_diff(time.ticks_ms(), last_valve_command_ms)
162                     >= COMMAND_TIMEOUT_MS):
163                 valve_off()
164                 print("Valve LED OFF: command timeout")
165             client.check_msg()
166             now = time.ticks_ms()
167             if time.ticks_diff(now, last_read_ms) >= READ_INTERVAL_MS:
168                 last_read_ms = now
169                 reading_number += 1
170                 try:
171                     sensor.measure()
172                     temperature = sensor.temperature()
173                     humidity = sensor.humidity()
174                 except OSError as error:
175                     print("Reading", reading_number, "| SENSOR ERROR:", error)
176                     client.ping()
177                 else:
178                     client.publish(MQTT_TOPIC, str(temperature).encode(),
    retain=False)
179                     client.publish(HUMIDITY_TOPIC, str(humidity).encode(),
    retain=False)
180                     print(
181                         "Reading", reading_number,
182                         "| Temperature published:", temperature, "C",
183                         "| Humidity published:", humidity, "%"
184                     )
185         except Exception as error:
186             print("MQTT communication ERROR:", repr(error))
187             close_mqtt()
188             retry_pause()
189             continue
190         time.sleep_ms(100)
```

Anhang A: ESP32-Firmware

Vollständige Firmware aus firmware/esp32_bms_step2.py. Zugangsdaten bleiben leer und werden lokal ergänzt. Umgebrochene Zeilen dienen nur der Lesbarkeit des Berichts.

```
except KeyboardInterrupt:
    print("Program stopped manually")
finally:
    close_mqtt()
    print("Fan OFF | Valve LED OFF | MQTT connection closed")

```

Anhang B: Node-RED Functions

Originale Function-Inhalte des aktuellen Exports. Alle 24 Funktionen sind enthalten; Verdrahtung und Node-Konfigurationen stehen in flows/bms-step2-flow.json.

```
// FUNCTION: Prepare MQTT Temperature
if (typeof msg.payload !== "number" ||
    !Number.isFinite(msg.payload)) {
    node.warn("Invalid temperature");
    return null;
}

msg.topic = "Temperature Istwert";
return msg;

// FUNCTION: Cooling Hysteresis
let temperature = msg.payload;

let setpoint = flow.get("setpoint");
let coolingState = flow.get("coolingState");

if (setpoint === undefined) {
    setpoint = 23;
}

if (coolingState === undefined) {
    coolingState = false;
}

let onThreshold = setpoint + 1;
let offThreshold = setpoint - 1;

if (temperature >= onThreshold) {
    coolingState = true;
}

if (temperature <= offThreshold) {
    coolingState = false;
}

flow.set("coolingState", coolingState);

msg.setpoint = setpoint;
msg.onThreshold = onThreshold;
msg.offThreshold = offThreshold;
msg.coolingDemand = coolingState;

return msg;

// FUNCTION: Fan Alarm Logic
let autoMode = flow.get("autoMode");
let manualFanCommand = flow.get("manualFanCommand");
```

Anhang B: Node-RED Functions

Originale Function-Inhalte des aktuellen Exports. Alle 24 Funktionen sind enthalten; Verdrahtung und Node-Konfigurationen stehen in flows/bms-step2-flow.json.

```
let fanFeedback = flow.get("fanFeedback");

if (autoMode === undefined) {
    autoMode = true;
}

if (manualFanCommand === undefined) {
    manualFanCommand = false;
}

if (fanFeedback === undefined) {
    fanFeedback = false;
}

// Existing operating-mode logic
const dataTimeout = msg.dataTimeout === true;

if (autoMode === true) {
    msg.operationMode = "AUTO";

    if (dataTimeout) {
        msg.fanCommand = false;
    } else {
        msg.fanCommand = msg.coolingDemand === true;
    }

} else {
    msg.operationMode = "MANUAL";
    msg.fanCommand = manualFanCommand === true;
}

msg.fanFeedback = fanFeedback;

// Fan feedback monitoring
const feedbackDelayMs = 5000;
const now = Date.now();

let feedbackMissingSince = context.get("feedbackMissingSince");

if (msg.fanCommand === true && fanFeedback === false) {

    // Start timing when missing feedback is first detected.
    if (feedbackMissingSince === undefined) {
        feedbackMissingSince = now;
        context.set("feedbackMissingSince", feedbackMissingSince);
    }
```

Anhang B: Node-RED Functions

Originale Function-Inhalte des aktuellen Exports. Alle 24 Funktionen sind enthalten; Verdrahtung und Node-Konfigurationen stehen in flows/bms-step2-flow.json.

```
    const elapsedMs = now - feedbackMissingSince;

    msg.fanStartPending = elapsedMs < feedbackDelayMs;

    if (elapsedMs >= feedbackDelayMs) {
        msg.alarm = true;
        msg.alarmText = "Fan fault: Command ON but no feedback";

    } else {
        msg.alarm = false;
        msg.alarmText = "Waiting for fan feedback";
    }

} else {
    // Command OFF or feedback received: reset the monitoring.
    context.set("feedbackMissingSince", undefined);

    msg.fanStartPending = false;
    msg.alarm = false;
    msg.alarmText = "No alarm";
}

return msg;

// FUNCTION: Set Fan Status
flow.set("fanHealthy", msg.payload);
return msg;

// FUNCTION: Set Operation Mode
flow.set("autoMode", msg.payload);
return msg;

// FUNCTION: Set Manual Fan Command
flow.set("manualFanCommand", msg.payload);
return msg;

// FUNCTION: Set Temperature Setpoint
flow.set("setpoint", msg.payload);
return msg;

// FUNCTION: Write Sim Temperature
msg.payload = {
    value: 246,
    register: "input",
    address: 0,
    disableMsgOutput: 0
};
```

Anhang B: Node-RED Functions

Originale Function-Inhalte des aktuellen Exports. Alle 24 Funktionen sind enthalten; Verdrahtung und Node-Konfigurationen stehen in flows/bms-step2-flow.json.

```
return msg;

// FUNCTION: Scale Setpoint for Modbus
msg.payload = msg.payload * 10;
return msg;

// FUNCTION: Scale Read Setpoint
let setpoint = flow.get("setpoint");
if (setpoint === undefined || setpoint === 0) {
    flow.set("setpoint", 23);
}

if (!Array.isArray(msg.payload) || msg.payload.length === 0) {
    return null;
}

const rawValue = Number(msg.payload[0]);
if (!Number.isFinite(rawValue) || rawValue === 0) {
    return null;
}

msg.payload = rawValue * 0.1;
flow.set("setpoint", msg.payload);
msg.topic = "Setpoint";
return msg;

// FUNCTION: Prepare Fan Command
msg.payload = msg.fanCommand;
return msg;

// FUNCTION: Set Fan Feedback
msg.payload = {
    value: msg.payload,
    register: "discrete",
    address: 0,
    disableMsgOutput: 0
};

return msg;

// FUNCTION: Store Fan Feedback
let feedback;

if (Array.isArray(msg.payload)) {
    feedback = msg.payload[0];
} else {
```

Anhang B: Node-RED Functions

Originale Function-Inhalte des aktuellen Exports. Alle 24 Funktionen sind enthalten; Verdrahtung und Node-Konfigurationen stehen in flows/bms-step2-flow.json.

```
    feedback = msg.payload;
}

flow.set("fanFeedback", feedback);

return msg;

// FUNCTION: Chart: payload + topic only
// Keep Modbus buffers and transport metadata out of chart history.
// The control branches retain their original complete messages.
return { payload: msg.payload, topic: msg.topic };

// FUNCTION: Fan Command to Text
msg.payload = msg.fanCommand ? "ON" : "OFF";
return msg;

// FUNCTION: Fan Feedback to Text
msg.payload = msg.fanFeedback ? "ON" : "OFF";
return msg;

// FUNCTION: Alarm to Text
if (msg.alarm === true) {
    msg.payload = "ALARM - Fan fault";
} else {
    msg.payload = "No alarm";
}

return msg;

// FUNCTION: Prepare MQTT Fan Command
// Convert the fan command to an MQTT message
msg.payload = msg.fanCommand === true ? "ON" : "OFF";
msg.topic = "bms/room1/fan/command";

return msg;

// FUNCTION: Prepare Humidity
const humidity = Number(msg.payload);

if (!Number.isFinite(humidity) ||
    humidity < 0 ||
    humidity > 100) {
    return null;
}

return {
    payload: humidity,
```

Anhang B: Node-RED Functions

Originale Function-Inhalte des aktuellen Exports. Alle 24 Funktionen sind enthalten; Verdrahtung und Node-Konfigurationen stehen in flows/bms-step2-flow.json.

```
    topic: "Relative Luftfeuchtigkeit"
};

// FUNCTION: Kühlventil Command
// Virtual cooling valve command
const coolingDemand = msg.coolingDemand === true;
const fanCommand = msg.fanCommand === true;
const fanFeedback = msg.fanFeedback === true;
const dataTimeout = msg.dataTimeout === true;

msg.valveCommand =
    !dataTimeout &&
    coolingDemand &&
    fanCommand &&
    fanFeedback;

msg.valveStatus = msg.valveCommand
    ? "OFFEN"
    : "GESCHLOSSEN";

return msg;

// FUNCTION: Fan Command Runtime
const now = Date.now();

// Keep the accumulated total on disk.
let totalMs = context.get("totalMs", "file") || 0;

// Keep timing and command state in memory only.
const lastUpdate = context.get("lastUpdate");
const previousCommand = context.get("previousCommand");

if (lastUpdate !== undefined && previousCommand === true) {
    const elapsed = now - lastUpdate;

    if (elapsed > 0 && elapsed <= 5000) {
        totalMs += elapsed;
    }
}

context.set("totalMs", totalMs, "file");
context.set("lastUpdate", now);
context.set("previousCommand", msg.fanCommand === true);

msg.fanCommandSeconds = Math.floor(totalMs / 1000);
msg.fanCommandHours = totalMs / 3600000;
```

Anhang B: Node-RED Functions

Originale Function-Inhalte des aktuellen Exports. Alle 24 Funktionen sind enthalten; Verdrahtung und Node-Konfigurationen stehen in flows/bms-step2-flow.json.

```
return msg;

// FUNCTION: Prepare MQTT Valve Command
msg.payload = msg.valveCommand === true ? "ON" : "OFF";
msg.topic = "bms/room1/valve/command";

return msg;

// FUNCTION: Fan Alarm History
// Process only messages containing an evaluated alarm state.
if (typeof msg.alarm !== "boolean") {
    return null;
}

// Restore the last three alarm records from disk.
let history = context.get("fanAlarmHistory", "file");

if (!Array.isArray(history)) {
    history = [];
}

const activeRecord = history.find(
    record => record.status === "AKTIV"
);

const timestamp = new Date().toLocaleString("de-DE", {
    timeZone: "Europe/Berlin"
});

let changed = false;

if (msg.alarm === true && !activeRecord) {
    // New alarm: create one record only.
    history.unshift({
        raisedAt: timestamp,
        clearedAt: "",
        status: "AKTIV",
        text: "Lüfter: Keine Rückmeldung bei Einschaltbefehl"
    });

    history = history.slice(0, 3);
    changed = true;

} else if (msg.alarm === false && activeRecord) {
    // Alarm condition ended: update the existing record.
    activeRecord.clearedAt = timestamp;
    activeRecord.status = "GEGANGEN";
```

Anhang B: Node-RED Functions

Originale Function-Inhalte des aktuellen Exports. Alle 24 Funktionen sind enthalten; Verdrahtung und Node-Konfigurationen stehen in flows/bms-step2-flow.json.

```
    changed = true;
}

// Write only when an alarm appears or clears.
if (changed) {
    context.set("fanAlarmHistory", history, "file");
}

msg.payload = history;
msg.topic = "Fan Alarm History";

return msg;

// FUNCTION: Fan Energy Estimate
// Assumed power for learning, not a measured value.
const assumedPowerW = 2;
const hours = msg.fanCommandHours;

if (typeof hours !== "number" ||
    !Number.isFinite(hours) || hours < 0) {
    return null;
}

msg.fanPowerAssumedW = assumedPowerW;
msg.fanEnergyEstimateWh = hours * assumedPowerW;
msg.payload = msg.fanEnergyEstimateWh.toFixed(3) + " Wh";
return msg;
```